

LAB 3: Implementing inheritance - Documentation

The aim of this lab is to implement the design of Seminar 3, which focuses on building an application for linear regression using object-oriented programming concepts. We extended Lab 2 by adding new classes: *Model*, *Algorithm*, and *SupervisedLearner* and modifying the existing *Vector* class to include an *augment()* method. We also created a *TestLearner* class to verify that all the new components interact correctly.

The program as a whole should be able to represent a dataset, run a gradient descent algorithm to learn a linear regression model, and then predict outputs for new inputs.

Classes, attributes and methods

We had to update:

1. **Vector:** Added the method *augment()* → *Vector* that returns a new vector with an additional 1 at the end to represent the bias term for linear regression.

We also had to define these new classes:

2. **Model:** Represents the parameter vector θ of a linear model.
 - **Attributes:**
 - *Vector params*: parameter vector of dimension $d + 1$, where d is the input dimension.
 - **Methods:**
 - *Model(int dim)*: constructor, initializes θ to zero
 - *getParams()* → *Vector*: returns the parameter vector.
 - *predict(Vector x)* → *double*: returns the scalar prediction $y^{\wedge} = \theta \cdot x$ (x must be augmented)
 - *update(Vector gradient, double alpha)*: updates the model parameters according to gradient descent $\theta = \theta - \alpha \nabla L(\theta)$
 - *toString()* → *String*: converts the model parameters to string for printing
3. **Algorithm:** Implements gradient descent for linear regression.
 - **Attributes:**
 - *double learningRate*: controls the step size
 - *double stoppingCriterion*: small threshold that determines when to stop iterating
 - **Methods:**
 - *Algorithm(double lr, double sc)*: constructor
 - *getLearningRate()* → *double*: returns learningRate
 - *getStoppingCriterion()* → *double*: returns stoppingCriterion
 - *gradient(Dataset ds, Model m)* → *Vector*: computes the gradient vector according to the formula from Seminar 3.
 - *solve(Dataset ds)* → *Model*: performs gradient descent by repeatedly computing the gradient and updating the model until convergence, and returns the learned Model.
4. **SupervisedLearner:** Manages all components of the learning process: the Dataset, the Algorithm, and the Model.
 - **Attributes:**
 - *Algorithm algorithm*: aggregation
 - *Dataset dataset*: aggregation
 - *Model model*: composition, created inside this class
 - **Methods:**
 - *SupervisedLearner(Algorithm a, Dataset d)*: constructor

- *getAlgorithm()* → *Algorithm*: returns the attribute algorithm
- *getDataset()* → *Dataset*: returns the parameter dataset
- *getModel()* → *Model*: returns the attribute model
- *solve()*: delegates to the algorithm's *solve()* and stores the resulting model.
- *predict(Vector v)* → *double*: augments the non-augmented input vector and returns the prediction using the current model.
- *toString()* → *String*: returns a readable string of the model parameters (or a message if untrained).
- *mp()* → *Model*: computes the Moore-Penrose inverse (returns null as we could not correctly import EJML)

5. TestLearner:

- Tests all the components by building a dataset, creating a learner, calling *solve()*, and verifying that predictions and model parameters behave as expected.
- Follows the same “works / does not work” style as previous labs, printing each test result and counting errors.

During the design phase, we discussed several possible implementations for the new classes and their relationships. One alternative was to implement gradient descent directly inside *SupervisedLearner*. This would have made the learner more self-contained but less flexible, since it would no longer be possible to reuse different optimization algorithms with the same learner. We therefore decided to let *SupervisedLearner* delegate to *Algorithm.solve()*, following a delegation pattern and emphasizing composition over inheritance.

We also discussed whether to pass an augmented dataset to the algorithm instead of augmenting each input on demand. We decided to augment inside the *predict()* and *gradient()* methods to keep the dataset independent of the regression implementation and reusable for other learners.

The theoretical object-oriented programming concepts we used for this lab are:

- **Encapsulation** is used throughout the lab, all attributes are private and accessed only through getters or controlled updates.
- **Composition and aggregation** relationships are respected: *SupervisedLearner* *owns* its *Model* (composition), but only *aggregates* the *Algorithm* and *Dataset*, which can exist independently.
- **Delegation** and **abstraction** are used to make each class handle only its part of the process.
- **Polymorphism** was not required in the lab, but inheritance appears when reusing *Dataset* and its specialized versions (*StandardizedDataset*, *NormalizedDataset*) from Lab 2.

The final implementation worked well in practice. The *TestLearner* class confirmed that all methods behaved correctly:

- First, we check in the *Vector* class that **augment()** correctly adds a dimension.
- Then, we test **gradient()** from the *Algorithm* class to verify that it does not return a zero norm.
- We also test **update()** from the *Model* class, which shows to complete the right computations and updates.
- Finally, we test the methods from the *SupervisedLearner* class. To do so, we create a 2-dimension dataset of four elements and linear relationship $y = x_1 + 2 \cdot x_2 + 1$.

- First, calling **toString()** before training the model correctly displays *Untrained model (call solve() first)*.
- Then, we check that **solve()** (which needs the solve() method from Algorithm to work) does train the model.
- We also check the method **predict()** from SupervisedLearner (needs predict() from Model), which works correctly (taking into account the tolerance) for non-augmented input vectors, since the method internally performs the augmentation step.
- In the end, **toString()** printed the correct values (with some tolerance), which shows that our implementation works.
- Additional: we wrote the comparison test for the Gradient and the **Moore-Penrose inverse**, but skip it as our Model is *null*.

We encountered a few difficulties and doubts while implementing this lab.

The most confusing part at first was understanding the relations between the classes, especially why SupervisedLearner uses composition with Model but aggregation with Dataset and Algorithm. After reading the lab instructions and Seminar 3 notes again, we realized that the learner must create and own its Model, while the dataset and algorithm can exist on their own.

Another challenge was dealing with floating-point precision when checking predictions. Because of small numerical errors, values that should have been 0 sometimes appeared as very tiny numbers like 1e-12. To avoid incorrect test failures, we added a small tolerance when comparing numbers, so two values count as equal if the difference is very small.

We also initially wondered what learning rate (α) and tolerance values to choose. After testing different combinations, we found that a learning rate of 0.01 and a tolerance of 0.000001 gave good results.

Additionally, we had some issues implementing the optional Moore-Penrose installing the EJML library despite attempting both the JAR-based setup and Gradle integration. As we were not able to implement it but we had already dedicated some time to it, we left some ideas and part of the implementation in the comments, as to let the compulsory part of the project work, while showing our comprehension of the additional part.

Finally, implementing the SupervisedLearner.predict() method required paying attention to augmenting the vector manually. Forgetting this step caused wrong predictions initially, until we added Vector.augment() inside the method.

Overall, the solution passed all tests with 0 errors. The classes integrated perfectly with those from both previous labs. This lab helped us understand how composition, delegation and abstraction work in practice and showed us how theoretical design concepts translate into an actual learning algorithm in Java.